features are missing for easily adding fault tolerance to the application:

- Re-creating objects and their dependencies and replacing these in the application structure isn't available as a first-class feature.
- Objects communicate with each other directly, so it's hard to isolate them.
- The fault-recovery code and the functional code are tangled with each other.

Luckily, we have a simpler solution. You've already seen some actor features that can help with these problems. Actors can be (re-)created from `Behaviors` functions and communicate through actor references instead of direct references. In the next section, we look at how actors provide a way to untangle the functional code from the fault-recovery code and how the actor lifecycle makes it possible to suspend and restart actors (without invoking the wrath of the concurrency gods) in the course of recovering from faults.

## *5.1.2 Wrap it up and let it crash*

In the previous section, you saw that building a fault-tolerant application with plain old objects and exception handling is complex. Let's look at how actors simplify this task. What should happen when an actor processes a message and encounters an exception? We have discussed why you don't want to graft recovery code into the operational flow, so catching the exception where the business logic resides is not an option.

Instead of catching exceptions in an actor, you let the actor crash. The actor code for handling messages contains only happy-path